

Unit – III

**Software Project Estimation and
Scheduling**

By: Yagnik Hathaliya

Major Learning Outcomes

- **Coding standards and guidelines**
- **Code Review**
 - Code Work Through
 - Code Inspection
- **Software Documentation**
 - Internal Documentation
 - External Documentation
- **Testing**
 - Testing Fundamentals
 - Functional Testing (Black Box Testing)
 - Structural Testing (White Box Testing)
 - Alpha Testing
 - Beta Testing
 - Unit Testing
 - Integration Testing
 - Test Documentation – Test Case Template

Code Standards and Guidelines

- **Coding** is undertaken once the design phase is complete and the design document has been successfully reviewed. In the coding phase, every module identified and specified in the design document is independently Coded and Tested.
- **Input to the Coding phase is Design Document.**
- **Objective of the Coding phase is to transform the design of a system into a high-level language code and to unit test this code.**
- Good software development organizations normally require their programmers to some well-defined and standard style of coding called Coding Standards.
- The Purpose of Coding Standards :
 - It gives a uniform appearance to the codes
 - It enhances code understanding
 - It encourages good programming practices
- **Coding Standards list several rules to be followed and Coding Guidelines provide general suggestions.**

- Good software development organizations usually develop their own coding standards and guidelines depending on what best suits their organization and the type of products they develop.
- *Coding Standards:*
 - The use of global should be limited.
 - Contents of the headers preceding codes for different modules
 - Name of the module
 - Date in which module is created
 - Author's Name and Details
 - Different functions supported along with their input/output parameters.
 - Global variables accessed/modified by the module.
 - Naming Conventions for global variables, local variables, and constant identifiers (Possible naming convention can be that global variable names always start with a capital letter, local variable names are made of small letters, and constant names are always capital letters)
 - Error Return Conventions and exception handling mechanisms (The way error conditions are reported by different functions in a program are handled should be standard within an organization.)

- *Coding Guidelines:*

- Do not use a coding style that is too clever or too difficult to understand
- Avoid unclear side effects (Any part of code should not be affecting the other part of the code.)
- Do not use an identifier for multiple purposes (Programmers often use the same identifier to denote some temporary entities, use of a variable for multiple purposes can lead to confusion and make it difficult for somebody trying to read and understand the code.)
- Each variable should be given a descriptive name indicating its purpose
- Use of variables for multiple purposes usually makes future enhancements more difficult.
- The code should be well-documented
- The length of any function should not exceed 10 source lines
- Do not use goto statements (Use of goto statements makes a program unstructured and makes it very difficult to understand.)
- Avoid Deep Nesting Structure.

- *Characteristics of Good Coding:*
 - Simplicity (Code Should be simple and Easy to Understand)
 - Readability (Code Should easily readable)
 - Good Documentation (Code Should be properly documented)
 - Usability (Code Should have error recovery techniques)
 - Transportability (Code Should be easy to transport in different environments)

Code Review

- Code Review is the systematic examination of source code by one or more developers with the goal of identifying and correcting errors, improving code quality and ensuring that the code conforms to best practices and standards, It is a Kind of verification.
- Code Review for a model is carried out after the module is successfully compiled and the all the syntax errors have been eliminated.
- Classification of Review:
 - **Formal Review** (Conducted at the end of each cycle/phase)
 - **Informal Review** (Conducted on as-needed basic)
- Advantages of Code Review:
 - Improves Product Quality
 - Improves Code Readability
 - Improves Estimation and Planning of the project
- Review Techniques are:
 - **Code Walk Through**
 - **Code Inspection**

- *Code Walk Through:*

- Code Walkthrough is an informal code review process where developers systematically go through the source code to find defects, improve quality, and ensure compliance with coding standards, that helps in early defect detection and knowledge sharing.
- A few members of the development team are given the code few days before the walk-through meeting to read and understand code.
- Each member selects some test cases and simulates execution of the code by hand (i.e. trace execution through each statement and function execution).
- **Objective : To discover the algorithmic and logical errors in the code.**
- The members note down their findings to discuss these in a walkthrough meeting where the coder of the module is present.
- Some of the guidelines for Code Walk through are the following.
 - The team performing code walk through should not be either too big or too small, it should consist of between three to seven members.
 - Discussion should focus on discovery of errors and not on how to fix the discovered errors.
 - Managers should not attend the walk-through meetings.

- *Code Inspection:*

- Code Inspection is a formal code review process where a team systematically examines the source code to detect defects, improve quality, and ensure compliance with coding standards before execution.
- It focuses on error prevention rather than debugging.
- Good software development companies collect statistics regarding different types of errors commonly committed by their engineers and identify the type of errors most frequently committed.
- Such a list of commonly committed errors can be used during code inspection to look out for possible errors.
- Following is a list of some classical programming errors which can be checked during code inspection:
 - Use of uninitialized variables.
 - Jumps into loops.
 - Non terminating loops.
 - Array indices out of bounds.
 - Improper storage allocation and deallocation.
 - Mismatches between actual and formal parameter in procedure calls.
 - Use of incorrect logical operators.
 - Improper modification of loop variables.

Software Documentation

- Software Documentation is the process of creating, maintaining, and organizing written materials that describe how software is designed, developed, and used, It helps developers, testers, users, and stakeholders understand the software system and its functionalities.
- When various kinds of software products are developed then not only the executable files and the source code are developed but also various kinds of documents such as;
 - Users' Manual, SRS Documents, Design Documents, Test Documents, Installation Manual etc.
- Good documents are very useful and used for the following purposes:
 - Enhance Understandability and Maintainability of a software product.
 - Reduce the effort and time required for maintenance.
 - Production of good documents helps the manager in effectively tracking the progress of the project.
 - Reduce Risk
- There are two types of Software Documentation;
 - Internal Documentation
 - External Documentation

- *Internal Documentation:*

- Internal documentation is the code understanding features provided as part of the source code itself and it is provided through appropriate module headers and comments in the source code.
- It is also provided through the useful variable names, comments, module and function headers, code structuring, use of enumerated types and constant identifiers, use of user-defined data types, etc.
- Good software development organizations usually maintain good internal documentation.

- *External Documentation:*

- External Documentation is provided through various types of supporting documents such as users' manual, software requirements specification document, design document, test documents, etc.
- Overall, Good External documentation plays an important role to ensure software understandability, maintainability and usability to both developers and end users.

Testing

- Testing is a crucial phase in software development that ensures the software meets requirements and functions correctly.
- Testing a program consists of providing the program with a set of Test Inputs (or Test Cases) and Observing if the program behaves as expected or not.
- If program fails to behave as expected, it need to be debugged and corrected.
- Aim of Testing:
 - Ensure the quality, reliability, and functionality of a software application before its release.
 - Identify all defects existing in a software product.
- Some commonly used terms associated with testing are:
 - Error: Mistake
 - Bug: Mistake done by Programmer at the time of coding
 - Failure: This is a demonstration of an error (or defect or bug).
 - Test Case: This is the triplet [I,S,O], where I is the data input to the system, S is the state of the system at which the data is input, and O is the expected output of the system.
 - Test Suite: This is the Set of all Test Cases with which a given software product is to be tested.

- *Differentiate Verification and Validation:*
 - **Verification** is the process of determining whether the output of one phase of software development confirms to that of its previous phase.
 - **Validation** is the process of determining whether a fully developed system confirms to its requirements specification.
 - Thus, verification is concerned with phase containment of errors and the Aim of validation is that the final product be error free.
 - **Verification: Are we doing right?**
 - **Validation: Have we done right?**

- *Design of Test Cases:*

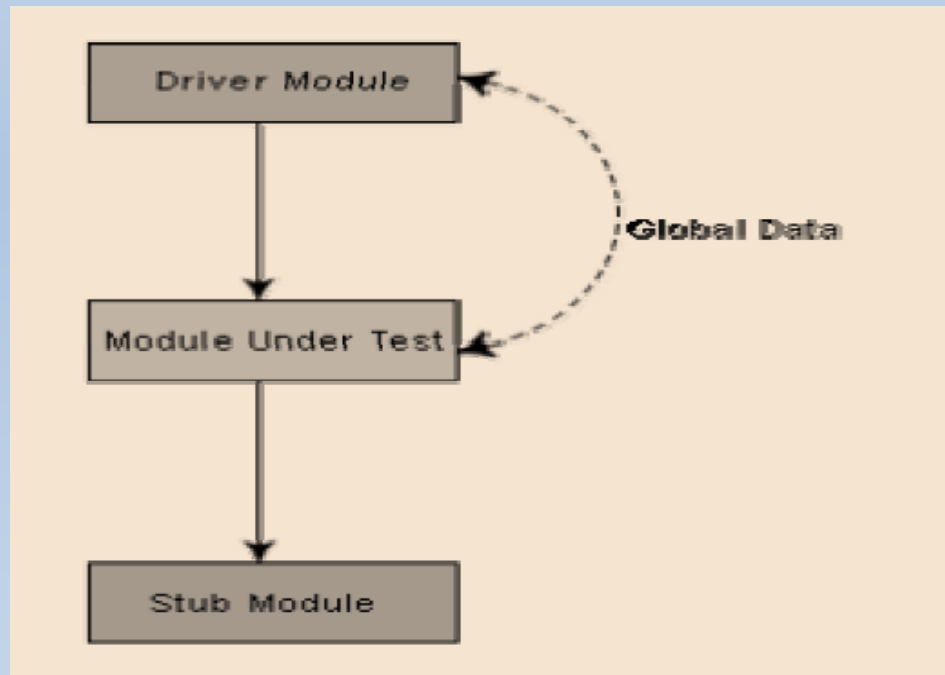
- We must design an optional test suite that is of reasonable size and can uncover as many errors existing in the system as possible.
- Testing a system using a large collection of test cases that are selected at random does not guarantee that all of the errors in the system will be uncovered.
- Systematic approach should be followed to design an optimal test suit, each test case is design to detect different errors.
- There are mainly Two Approaches to systematically design test cases:
- In the Black-Box Testing approach, test cases are designed using only the functional specification of the software, i.e. without any knowledge of the internal structure of the software, hence black-box testing is known as Functional Testing
- In the White-Box Testing approach, designing test cases requires thorough knowledge about the internal structure of software, hence the white-box testing is called Structural Testing

- *Testing in the Large vs. Testing in the Small:*

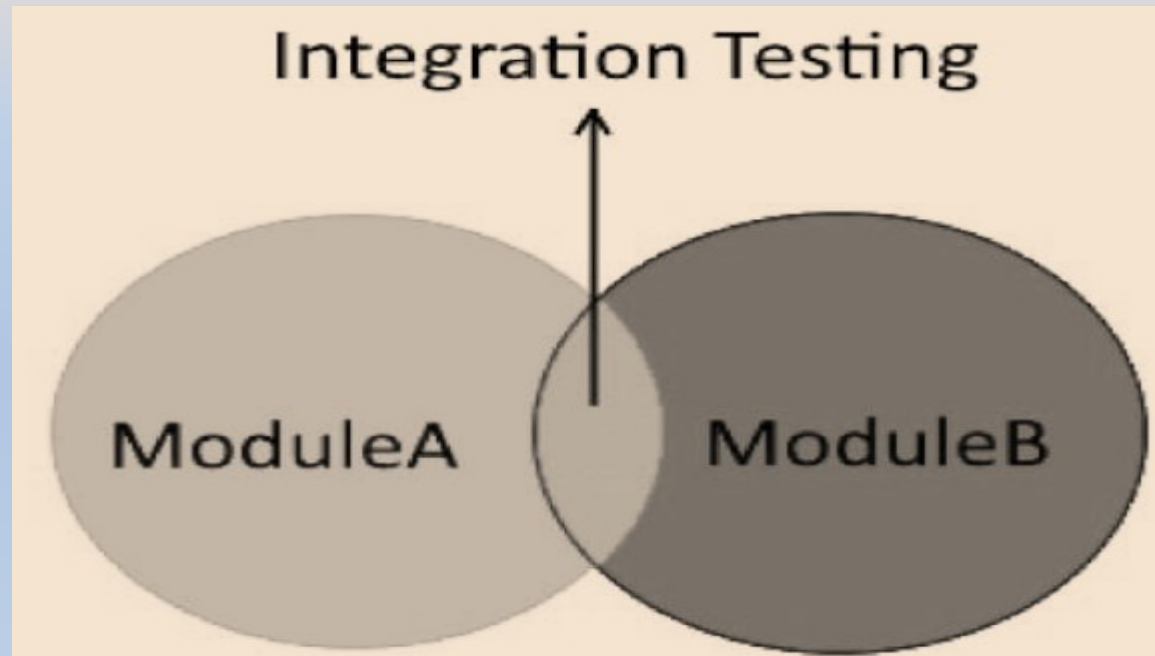
- Software products are normally tested first at the individual component or unit level, This is referred to as testing in the small.
- After testing all the components individually, the components are slowly integrated and tested at each level of integration (integration testing).
- Finally, the fully integrated system is tested (called system testing), Integration and system testing are known as testing in the large.

- **Unit Testing:**

- It is a First level of Testing, Unit Testing is undertaken after a module has been coded and successfully reviewed.
- Unit Testing (Module Testing) is the testing of different units (or modules) of a system in isolation.
- Stub and Driver Module provide the complete environment for a module and in this method the Calling Module and Called Module should be unit tested.
- The Role of Stub Module and Driver Module is pictorially shown in the figure



- A Stub Module/Procedure is a dummy procedure that has the same input parameters as the given procedure but has a highly simplified behavior.
- It produce the expected behavior using simple table look up mechanism.
- A Driver Module contain the nonlocal data structures accessed by the module under test, and would also have the code to call the different functions of the module with appropriate parameter values.



- *Integration Testing:*

- Integration testing is the second level of the software testing process comes after unit testing. In this testing, units or individual components of the software are tested in a group.
- The focus of the integration testing level is to expose defects at the time of interaction between integrated components or units.
- Unit testing uses modules for testing purpose, and these modules are combined and tested in integration testing.
- The Software is developed with a number of software modules that are coded by different coders or programmers.
- The goal of integration testing is to check the correctness of communication among all the modules.
- Integration testing is a software testing technique that focuses on verifying the interactions and data exchange between different components or modules of a software application.
- The goal of integration testing is to identify any problems or bugs that arise when different components are combined and interact with each other.
- Integration testing is typically performed after unit testing and before system testing.
- It helps to identify and resolve integration issues early in the development cycle, reducing the risk of more severe and costly problems later on.

- *Black-Box Testing (Functional Testing):*

- This method is also called Behavioral Testing or Functional Testing.
- In the black-box testing, test cases are designed from an examination of the input/output values only and no knowledge of design, or code is required.
- The following are the two main approaches to designing black box test cases.
 - Equivalence Class Portioning (ECP)
 - Boundary Value Analysis (BVA)

➤ Equivalence Class Portioning (ECP)

- In it, the domain of input values to a program is partitioned into a set of equivalence classes, so the behavior of the program is similar for every input data belonging to the same equivalence class.
- Main Idea: If one Test Case in a class detects an error, all other test cases in the class would be expected to find same error.
- Two Steps are required to implement this techniques;
 1. Equivalence Classes are identified by partition data into valid and invalid class.
 2. Generate Test cases using equivalent classes.
- The following are some general guidelines for designing the equivalence classes:
 - If the input data values to a system can be specified by a range of values, then one valid and two invalid equivalence classes should be defined.
 - If the input data assumes values from a set of discrete members of some domain, then one equivalence class for valid input values and another equivalence class for invalid input values should be defined.
 - If the input value is Boolean, then one valid and one invalid equivalence classes should be defined.

— Example:

- For a software that computes the square root of an input integer which can assume values in the range of 0 to 5000, there are three equivalence classes:
 - The set of negative integers
 - The set of integers in the range of 0 and 5000
 - The integers larger than 5000.
 - Therefore, the test cases must include values for each of the three equivalence classes and a possible test set can be: {-5,500,6000}

➤ Boundary Value Analysis (BVA)

- Generally, errors are occurred at boundary of domains rather than center of domain.
- Reason: Test cases closer to boundary detect more errors.
- Selection of test cases in BVA done at the edge of the class.
- For example, programmers may improperly use `<` instead of `<=`, or conversely `<=` for `<`.
- Example: For a function that accepts integer values in the range of 0 and 100, then the test cases must include the following values: {0, - 1, 100, 101}.
- Advantages: Its very simple to understand, High efficiency
- Disadvantages: Need practice to identify equivalence classes

- *White-Box Testing (Structural Testing):*

- This method is concerned with testing the implementation of the program.
- Aim: To investigate the internal logic and structure of the code, that is why It is also known as structural testing.
- In this it is necessary for a tester to have a full knowledge of source code.
- The different White box testing are listed below,
 - Statement Coverage (Line Coverage)
 - Branch Coverage
 - Condition Coverage
 - Path Coverage
 - Data Flow Based Testing
 - Mutation Testing

➤ Statement Coverage (Line Coverage)

- The principal idea of the statement coverage strategy is that unless a statement is executed, it is very hard to determine if an error exists in that statement.
- Aim: To design test cases so that every statement in a program is executed at least once.
- However, executing some statement once and observing that it behaves properly for that input value is no guarantee that it will behave correctly for all input values.
- Example: Grater Number of X and Y

```
int x, y;  
if (x>y)  
    then x is greater  
else  
    y is greater
```

Test case should be:
{(5,4), (4,5)}

— Example:

```
int compute_gcd(x, y)
    int x, y;
    {
        while (x != y)
        {
            if (x > y) then
                x = x - y;
            else y = y - x;
        }
        return x;
    }
```

- By choosing the test set $\{(x=3, y=3), (x=4, y=3), (x=3, y=4)\}$, we can exercise the program such that all statements are executed at least once.

➤ Branch Coverage

- In it, test cases are designed to make each branch condition to assume true and false values in turn.
- Branch testing is also known as edge testing as in it, each edge of a program's control flow graph is traversed at least once.
- Branch testing guarantees statement coverage, so it is stronger testing strategy than statement coverage.
- Example: For GCD algorithm , the test cases for branch coverage can be $\{(x=3, y=3), (x=3, y=2), (x=4, y=3), (x=3, y=4)\}$.

➤ Condition Coverage

- In which, each component of a condition expression to assume both true and false.
- For example, in the conditional expression $((c1.and.c2).or.c3)$, the components $c1$, $c2$ and $c3$ are each made to assume both true and false values.
- Condition testing is stronger than branch testing.
- For a composite conditional expression of n components, for condition coverage, 2^n test cases are required.
- Thus, for condition coverage, the number of test cases increases exponentially with the number of component conditions.
- Therefore, a condition coverage-based testing technique is practical only if n (the number of conditions) is small.

➤ Path Coverage

- In it, test cases are designed such that all linearly independent paths in the program are executed at least once.
- A linearly independent path can be defined in terms of the control flow graph (CFG) of a program.
- Control Flow Graph (CFG) :
 - A control flow graph describes the sequence in which the different instructions of a program get executed. In other words, a control flow graph describes how the control flows through the program.
 - Path is a node and edge sequence from the starting node to a terminal node of the control flow graph of a program.
 - There can be more than one terminal node in a program.
 - A linearly independent path is any path through the program that introduces at least one new edge(new node) that is not included in any other linearly independent paths.
 - In order to draw the control flow graph of a program, all the statements of a program must be numbered first.
 - The different numbered statements serve as nodes of the control flow graph.
 - An edge from one node to another node exists if the execution of the statement representing the first node can result in the transfer of control to the other node.
 - The CFG for any program can be easily drawn by knowing how to represent the sequence, selection, and iteration type of statements in the CFG.
 - After all, a program is made up from these types of statements.
 - Examples of CFG are;

Sequence:

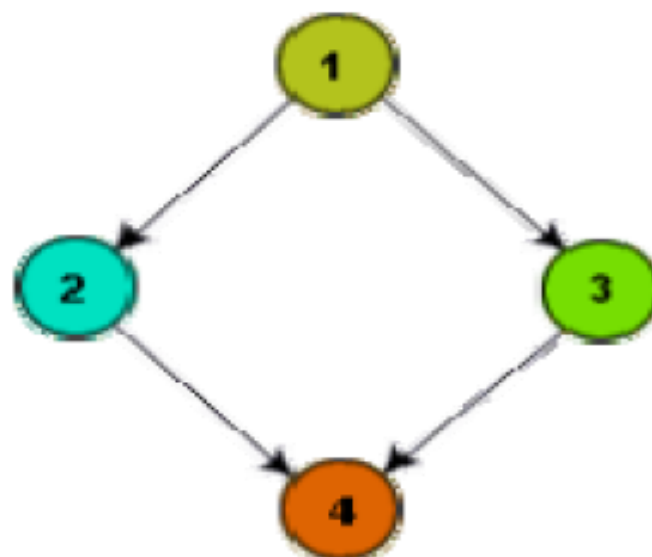
1. $a=5;$
2. $b=a^2-1;$



(a)

Selection:

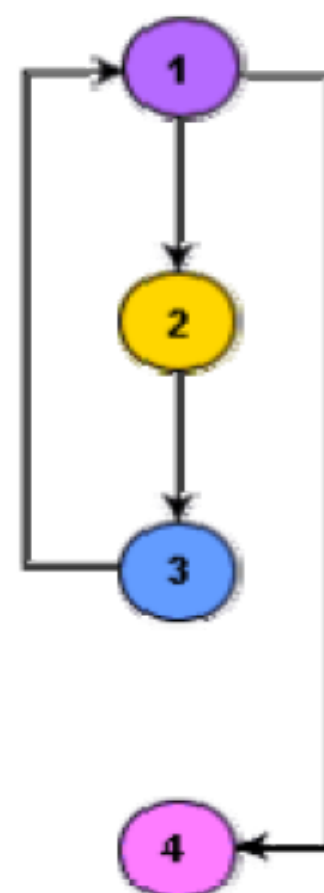
1. $\text{if}(a>b)$
2. $c=3;$
3. $\text{else } c=5;$
4. $c=c^*c;$



(b)

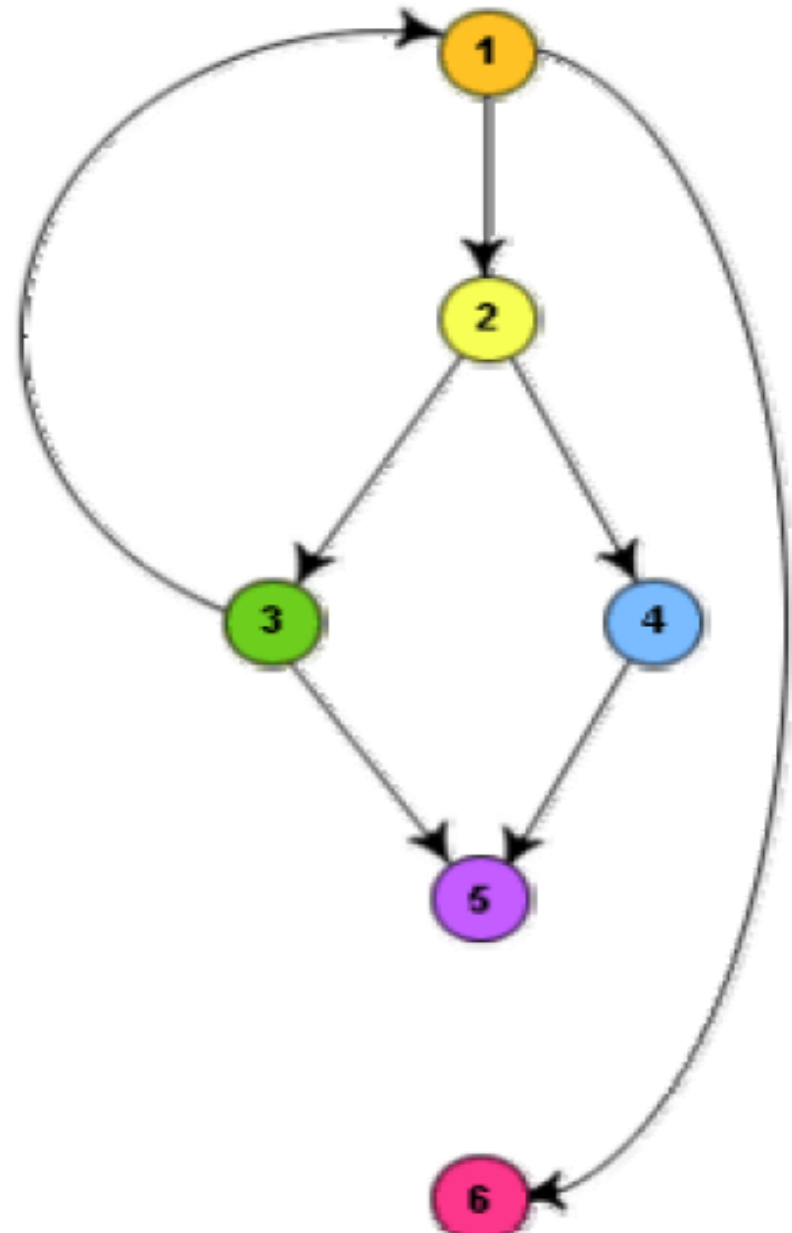
Iteration:

1. $\text{while}(a>b)\{$
2. $b=b-1;$
3. $b=b^*a;\}$
4. $c=a+b;$



(c)

```
int compute_gcd(int x, int y){  
1 while(x != y){  
2     if(x > y) then  
3         x = x - y;  
4     else y = y - x;  
5 }  
6 return x;  
}
```



– Cyclomatic Complexity

- McCabe's cyclomatic complexity defines an upper bound for the number of linearly independent paths through a program.
- It is very simple to compute.
- It is a practical way to determine maximum number of independent path in the program.
- There are three different ways to compute the cyclomatic complexity.
- **Method:1**
 - Given a CFG G of a program, the cyclomatic complexity $V(G)$ can be computed as;
 - $V(G) = E - N + 2$
 - Where N is the number of nodes of the control flow graph and E is the number of edges in the control flow graph.
 - For the CFG of above example, $E=7$ and $N=6$. Therefore, the cyclomatic complexity = $7-6+2 = 3$.

- Method:2

- Another way of computing the cyclomatic complexity $V(G)$ is
- $V(G) = \text{Total number of bounded areas} + 1$
- Any region enclosed by nodes and edges can be called as a bounded area.
- For the CFG of above example , number of bounded areas is 2. Therefore, the cyclomatic complexity, computing with this method is also $2+1 = 3$.

- Method:3

- The cyclomatic complexity of a program can also be easily computed by computing the number of decision statements of the program.
- $V(G) = \text{Total number of decision statements of the program} + 1$

➤ Data Flow Based Testing

- It selects test paths of a program according to the locations of the definitions and uses of different variables in a program.
- For a statement numbered S, let
$$\text{DEF}(S) = \{X/\text{statement } S \text{ contains a definition of } X\}, \text{ and}$$
$$\text{USES}(S) = \{X/\text{statement } S \text{ contains a use of } X\}$$
- Example:

For the statement $S:a=b+c;$

$$\text{DEF}(S) = \{a\}.$$
$$\text{USES}(S) = \{b,c\}$$

➤ Mutation Testing

- In it, software is first tested by initial test suit built from different white box techniques, then mutation testing is taken up.
- Main idea is to make few changes to program at a time.
- Each time a program is changed, it is called mutated program and change effected is called a mutant.
- Mutated program is tested against full test suit.
- If at least one test case in the test suit for which mutant gives an incorrect result, then the mutant is said to be dead. If a mutant remains alive even after all the test, the test data kill the mutant.

- *Alpha and Beta Testing:*

- **Alpha Testing** is a type of acceptance testing. performed to identify all possible issues and bugs before releasing the final product to the end users.
- Alpha testing is carried out by the testers who are internal employees of the organization.
- The main goal is to identify the tasks that a typical user might perform and test them.
- It is performed at the end of the development of the software, and before beta testing.
- The main focus of alpha testing is to simulate real users by using a black box and white box techniques.
- **Beta Testing** is performed by “real users” of the software application in “real environment” and it can be considered as a form of external User Acceptance Testing.
- It is the final test before shipping a product to the customers. Direct feedback from customers is a major advantage of Beta Testing.
- This testing helps to test products in customer’s environment.
- Beta version of the software is released to a limited number of end-users of the product to obtain feedback on the product quality.
- Beta testing reduces product failure risks and provides increased quality of the product through customer validation.

Alpha Testing	Beta Testing
Alpha testing performed by a team of highly skilled testers who are usually the internal employee of the organization.	Beta testing performed by clients or end-users in a real-time environment, who is not an employee of the organization.
Alpha testing performed at the developer's site; it always needs a testing environment or lab environment.	Beta testing doesn't need any lab environment or the testing environment; it is performed at a client's location or end-user of the product.
Reliability or security testing not performed in-depth in alpha testing.	Reliability, security, and robustness checked during beta testing.
Alpha testing involves both white box and black-box techniques.	Beta testing uses only black-box testing.
Critical issues or fixes can be identified by developers immediately in alpha testing.	Most of the issues or feedback is collecting from the beta testing will be implemented for the future versions of the product.
Alpha testing performed before the launch of the product into the market.	At the time of software product marketing.
Alpha testing performed nearly the end of the software development.	Beta testing is a final test before shipping a product to the customers.
Alpha testing is conducting in the presence of developers and the absence of end-users.	Beta testing reversed of alpha testing.

- *Test Documentation:*

- The documentation which is generated towards the end of testing is the test summary reports.
- It provides summary of test suits which has been applied to the system.
- It specify how many test suits are successful, how many are unsuccessful and what is the degree of successful and unsuccessful.
- It Includes following;
 - Test Design Specification
 - Test Case Specification
 - Test Procedure Specification
 - Test Item Transmittal Report
 - Test Log
 - Test Incident Report
 - Test Summary Report

- **Test Case Template:**

- A Test Case Template is a well-designed document for developing and better understanding of the test case data for a particular test case scenario.
- A good Test Case template maintains test artifact consistency for the test team and makes it easy for all stakeholders to understand the test cases.
- Writing test case in a standard format lessen the test effort and the error rate.

Test Case ID		BU_001	Test Case Description		Test the Login Functionality in Banking						
Created By		Mark	Reviewed By		Bill		Version		2.1		
QA Tester's Log		Review comments from Bill incorporate in version 2.1									
Tester's Name		Mark	Date Tested		1-Jan-2017		Test Case (Pass/Fail/Not		Pass		
S #	Prerequisites:						S #	Test Data			
1	Access to Chrome Browser						1	Userid = mg12345			
2							2	Pass = df12@434c			
3							3				
4							4				
Test Scenario		Verify on entering valid userid and password, the customer can login									
Step #	Step Details		Expected Results		Actual Results			Pass / Fail / Not executed / Suspended			
1	Navigate to http://xyz.com		Site should open		As Expected			Pass			
2	Enter Userid & Password		Credential can be entered		As Expected			Pass			
3	Click Submit		Cutomer is logged in		As Expected			Pass			